PROJET 02 — SÉCURITÉ DES APPLICATIONS WEB

Cahier des charges

Campus Portal — système sécurisé d’authentification et de cryptographie

Ce document décrit ce que l’application devra faire et ce qu’elle devra garantir, avant qu’une seule ligne n’en soit écrite. Il sert de référence pour l’évaluation du projet : chaque exigence fonctionnelle y porte un identifiant, et chaque identifiant trouve sa contrepartie dans le plan de tests de la section 9.

DOCUMENT	STATUT	DURÉE	VERSION
Cahier des charges	**Planification — non démarré**	**6 semaines**	**1.0**

SOMMAIRE

1. Présentation et objectif
2. Exigences techniques
3. Architecture de l’application
4. Périmètre du projet
5. Rôles et acteurs
6. Exigences fonctionnelles
7. Sécurité
8. Modèle de données
9. Plan de tests de sécurité
10. Planning en six semaines
11. Livrables
12. Critères de réussite et contraintes

1. Présentation et objectif

Le projet consiste à développer **Campus Portal**, une application web de gestion scolaire réunissant les comptes des utilisateurs, le catalogue des cours, les inscriptions, les notes, les certifications délivrées par l'établissement et une messagerie privée entre étudiants et enseignants.

L'application n'est pas une fin en soi : elle sert de support à l'objectif réel du projet, qui est d'appliquer correctement l'authentification, l'autorisation et la cryptographie, puis de pouvoir démontrer concrètement ce que chacune de ces protections empêche. Un comportement correct ne constitue pas une preuve en soi ; le projet devra donc produire des éléments vérifiables, rejouables devant la classe.

Le travail suit le découpage en six semaines imposé à l'ensemble des projets : **concevoir, implémenter, sécuriser, tester, améliorer, livrer**. La sécurité n'est donc pas traitée au fil de l'eau. L'application est d'abord développée dans sa version fonctionnelle, au cours des semaines 1 et 2, puis sécurisée pendant une phase dédiée en semaine 3. Ce découpage est volontaire : il permet de mesurer précisément l'apport de la phase de sécurisation, en comparant le comportement de l'application avant et après celle-ci.

2. Exigences techniques

Langage et exécution. L'application sera écrite en Python 3 et s'appuiera uniquement sur la bibliothèque standard, sans framework. Elle tournera en un seul processus, afin que chaque décision de sécurité reste lisible dans le code.

Base de données. La persistance sera assurée par SQLite, avec un fichier de base distinct pour chaque état de l'application. Le schéma sera déclaré dans un fichier SQL unique.

Interface. Le front-end sera écrit en HTML, CSS et JavaScript natifs, servi par l'application elle-même. Il consommera une interface JSON, ce qui rendra chaque action reproductible depuis un terminal ou un proxy.

Cryptographie. Le chiffrement au repos utilisera AES-256-GCM et les signatures de certifications Ed25519, via la bibliothèque cryptography. Les jetons de session et le hachage des mots de passe reposeront sur les modules hmac et hashlib de la bibliothèque standard.

Outillage de test. Burp Suite servira à intercepter, modifier et rejouer les requêtes, OpenSSL à manipuler les éléments cryptographiques. curl et les outils développeur du navigateur permettront de consigner chaque test sous une forme rejouable.

Reproductibilité. La base de données et les clés seront régénérées par une commande unique et documentée. L'exécution restera strictement locale.

3. Architecture de l’application

L’application repose sur un processus unique qui sert à la fois l’interface et les données. Le schéma ci-dessous situe la **frontière de confiance** : c’est la ligne que franchit chaque requête, et elle sépare ce qui est contrôlé par l’utilisateur de ce qui est contrôlé par le serveur.

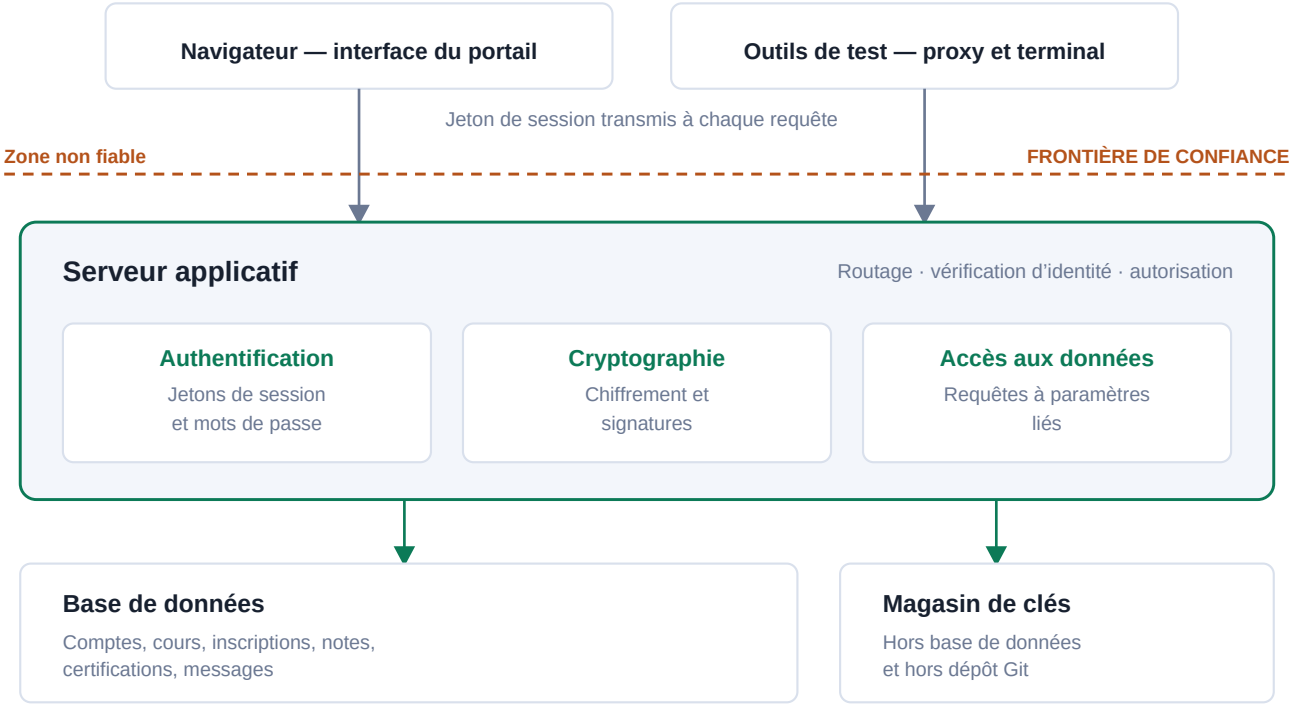

Figure 1 — Vue d’ensemble des composants et de la frontière de confiance.

La couche cliente regroupe le navigateur et les outils de test. Tout ce qui en provient — jeton de session, identifiants d’objets, paramètres d’URL, corps de requête, en-têtes — doit être considéré comme potentiellement forgé. Rien n’y est digne de confiance avant vérification.

Le serveur applicatif reçoit la requête, détermine le gestionnaire correspondant, établit l’identité de l’appelant à partir du jeton de session, puis applique la décision d’autorisation avant d’exécuter quoi que ce soit. Il s’appuie sur trois modules aux responsabilités séparées : l’**authentification** gère les jetons de session et le stockage des mots de passe ; la **cryptographie** assure le chiffrement des données sensibles et la signature des documents délivrés ; l’**accès aux données** centralise toutes les requêtes vers la base, avec les paramètres liés comme unique voie autorisée.

La persistance se compose de la base de données relationnelle et d’un magasin de clés distinct. Les clés cryptographiques ne sont jamais stockées en base ni versionnées dans le dépôt : elles sont générées au premier démarrage et conservées à part.

4. Périmètre du projet

Le périmètre couvre quatre ensembles fonctionnels, qui forment ensemble un portail utilisable de bout en bout.

Le cycle de vie des comptes comprend l'inscription, la connexion, la déconnexion et la gestion de la session, pour les trois rôles décrits en section 5.

Les données académiques regroupent le catalogue des cours, les inscriptions des étudiants, les notes saisies par les enseignants et les certifications délivrées par l'administration.

La messagerie privée permet des conversations individuelles entre deux comptes, quels que soient leurs rôles.

Les preuves de sécurité constituent le quatrième ensemble, au même titre que les fonctionnalités : un jeu de scénarios rejouables qui documentent le comportement de l'application avant et après la phase de sécurisation.

5. Rôles et acteurs

Trois rôles sont définis. Le rôle est une propriété du compte, enregistrée en base de données. Il sera relu depuis la base à chaque action privilégiée, et jamais déduit de la requête entrante. Un nouvel inscrit reçoit systématiquement le rôle étudiant : le choix du rôle n'appartient pas au client.

Rôle	Responsabilité	Actions permises
Étudiant	Consulte son propre dossier	Consulter les cours auxquels il est inscrit et les notes obtenues · consulter les certifications qui lui ont été délivrées · échanger des messages privés. Il n'a aucun accès au dossier d'un autre étudiant.
Enseignant	Responsable de ses cours	Consulter les cours dont il est titulaire et les étudiants qui y sont inscrits · saisir et modifier les notes de ces cours uniquement · échanger des messages privés.
Administrateur	Gère l'établissement	Créer et administrer l'ensemble des comptes · délivrer et signer les certifications · disposer d'une visibilité complète sur le portail.

6. Exigences fonctionnelles

Ces exigences décrivent le comportement attendu de l'application à l'issue de la phase d'implémentation, en semaine 2.

ID	Exigence	Description
EF-01	**Inscription**	Un visiteur crée un compte avec un identifiant et un mot de passe. Le rôle est attribué par le serveur.
EF-02	**Connexion**	Des identifiants valides ouvrent une session. Des identifiants invalides sont refusés sans préciser quel champ est en cause.
EF-03	**Déconnexion**	Le client abandonne sa session ; aucun état résiduel ne subsiste côté serveur.
EF-04	**Catalogue des cours**	Tout utilisateur connecté consulte la liste des cours et l'enseignant titulaire de chacun.
EF-05	**Mes cours**	Chaque rôle voit les cours qui le concernent : ceux auxquels il est inscrit, ceux dont il est titulaire, ou la totalité.

ID	Exigence	Description
EF-06	**Notes**	Un étudiant consulte ses notes. Un enseignant saisit une note sur un cours dont il est titulaire. Un administrateur peut faire les deux.
EF-07	**Certifications**	Un étudiant consulte les certifications qui lui ont été délivrées. Seul un administrateur peut en délivrer une.
EF-08	**Liste des conversations**	Un utilisateur voit les conversations auxquelles il participe, de la plus récente à la plus ancienne.
EF-09	**Lecture d’une conversation**	Un utilisateur ouvre une conversation à laquelle il appartient et en lit les messages dans l’ordre.
EF-10	**Envoi d’un message**	Un utilisateur adresse un message à un compte existant.
EF-11	**Annuaire des comptes**	Un administrateur liste l’ensemble des comptes. Aucun élément d’authentification ne figure dans la réponse.
EF-12	**Vérification publique**	La clé publique de l’émetteur est publiée, afin qu’un tiers puisse vérifier une certification sans accéder au serveur.

7. Sécurité

La sécurisation constitue une phase à part entière du projet, menée en **semaine 3**, une fois l'application fonctionnelle et testable. Elle n'est pas répartie dans le développement : elle intervient après lui, sur une base déjà complète. Les axes ci-dessous décrivent ce qui sera traité pendant cette phase et serviront de fil conducteur à la démonstration finale.

7.1 Sécurité de l'authentification

La session sera portée par un jeton signé à l'aide d'un code d'authentification de message. L'algorithme de signature sera imposé par le serveur et non lu dans le jeton ; la signature sera recalculée et comparée à chaque requête ; une durée de validité sera inscrite dans le jeton et contrôlée. Un jeton dont la signature ne correspond pas, dont l'algorithme n'est pas celui attendu, ou dont la validité est dépassée, sera refusé sans autre examen.

7.2 Contrôle d'accès et autorisation

Toute décision d'autorisation partira de l'état du serveur, jamais d'une affirmation du client. Deux niveaux seront traités. Le premier porte sur les **fonctions** : avant d'exécuter une action privilégiée, le serveur relit en base le rôle réel de l'appelant. Le second porte sur les **objets** : l'accès à un enregistrement n'est accordé qu'à son propriétaire, à son interlocuteur, ou à un administrateur. Un identifiant présent dans une requête ne confère à lui seul aucun droit — modifier un numéro dans une URL ne doit jamais ouvrir le dossier d'autrui.

7.3 Les entrées sont des données, jamais des instructions

Toute valeur venant de l'utilisateur sera transmise à la base de données en tant que paramètre lié, sur un canal distinct de celui de la requête. Aucune saisie ne sera concaténée dans une instruction. C'est la différence entre une donnée que la base interprète comme une valeur et une chaîne qu'elle interprète comme du code : la première ne peut pas modifier le sens de la requête, la seconde le peut. Le même principe s'appliquera à l'affichage, où le contenu produit par les utilisateurs sera traité comme du texte et non comme du balisage.

7.4 Stockage des mots de passe

Les mots de passe ne seront jamais conservés en clair, ni journalisés. Ils seront passés par une fonction de dérivation lente, avec un sel aléatoire propre à chaque compte, de sorte que deux comptes partageant le même mot de passe n'aient pas la même empreinte. La vérification se fera en temps constant, pour ne rien révéler par la durée de la réponse.

7.5 Confidentialité des données stockées

Le corps des messages privés sera chiffré avant son écriture en base, au moyen d'un chiffrement symétrique authentifié. Une personne qui obtiendrait le fichier de base de données n'y lirait donc rien d'exploitable. Ce chiffrement protège le fichier ; il ne remplace pas le contrôle d'accès de la section 7.2, puisque le serveur déchiffre avant de répondre : les deux protections sont complémentaires.

7.6 Intégrité et authenticité des documents délivrés

Chaque certification sera signée par l'émetteur au moyen d'une signature numérique asymétrique, calculée sur une représentation canonique de ses champs. Toute modification ultérieure d'un champ signé — note, intitulé, date, bénéficiaire — invalidera la signature, et l'application le signalera. La clé publique étant publiée, la vérification ne nécessite aucun accès privilégié.

7.7 Gestion des clés

Les clés seront générées au premier démarrage, conservées hors de la base de données et exclues du dépôt. Le secret repose sur la clé, jamais sur la discrétion du procédé : l'ensemble des mécanismes décrits ici peut être publié sans affaiblir le système.

8. Modèle de données

Six entités suffisent à couvrir le périmètre. Les **comptes** portent l'identité, les éléments d'authentification et le rôle qui commande toutes les décisions d'autorisation. Les **cours** constituent le catalogue, chacun rattaché à un enseignant titulaire unique. Les **inscriptions** indiquent quel étudiant suit quel cours. Les **notes** enregistrent une évaluation par étudiant et par cours, en conservant l'auteur de la saisie. Les **certifications** correspondent aux documents délivrés par l'administration, chacun accompagné de la signature de son émetteur. Les **messages** enfin portent les conversations individuelles, dont le contenu est protégé avant stockage.

Chaque build dispose de sa propre base de données, afin que la version fonctionnelle et la version sécurisée ne partagent jamais d'état et puissent être comparées directement. Le schéma détaillé accompagnera le code source.

9. Plan de tests de sécurité

Chaque test ci-dessous sera exécuté deux fois : une première fois sur l'application telle qu'issue de la phase d'implémentation, puis une seconde fois, à l'identique, après la phase de sécurisation. Un axe de la section 7 n'est considéré comme traité que lorsque les deux colonnes ont été observées et consignées.

ID	Test réalisé	Avant sécurisation	Après sécurisation	Axe
TS-01	**Connexion avec un identifiant forgé**	Session accordée	Refusé	7.3
TS-02	**Demande du dossier d'un autre étudiant**	Dossier renvoyé	Refusé	7.2 · 7.3
TS-03	**Présentation d'un jeton non signé**	Accepté	Refusé	7.1
TS-04	**Présentation d'un jeton expiré**	Accepté	Refusé	7.1
TS-05	**Ouverture de la conversation de deux tiers**	Affichée	Refusé	7.2
TS-06	**Inscription en réclamant le rôle administrateur**	Accordé	Rôle étudiant	7.2
TS-07	**Saisie d'une note par un étudiant**	Enregistrée	Refusé	7.2
TS-08	**Délivrance d'une certification par un étudiant**	Délivrée	Refusé	7.2
TS-09	**Modification d'une certification en base**	Reste valide	Signalée invalide	7.6
TS-10	**Lecture des messages dans le fichier de base**	Lisible	Illisible	7.5
TS-11	**Inspection des mots de passe stockés**	Réversible	Salé, non réversible	7.4

10. Planning en six semaines

Le projet suit le découpage commun à tous les projets. Chaque semaine produit un résultat concret ; la semaine 6 transforme le travail en projet démontrable et documenté.

Sem.	Phase	Travaux	Résultat attendu
1	Concevoir	Scénario, objectifs, architecture et mise en place	Cahier des charges validé, schéma de données, squelette de l’application
2	Implémenter	Développement et configuration des composants principaux	Portail fonctionnel : comptes, cours, notes, certifications, messagerie
3	Sécuriser	Contrôles de sécurité, configuration et protection	Les sept axes de la section 7 mis en œuvre sur l’application existante
4	Tester	Scénarios d’attaque, validation et tests	Plan de tests de la section 9 exécuté et consigné, avant et après
5	Améliorer	Analyse, correction, remédiation et optimisation	Écarts corrigés, comportement stabilisé, documentation mise à jour
6	Livrer	Preuves, rapport, présentation et démonstration	Dossier complet et démonstration en direct devant la classe

11. Livrables

Le code source et la base de données. L’application dans ses deux états, accompagnée du schéma et d’un jeu de données initial reproductible par une commande unique.

Le diagramme d’architecture. La carte des composants, la frontière de confiance et le modèle de données, sous la forme présentée en section 3.

Les cas de tests de sécurité. L’ensemble des scénarios de la section 9, avec la requête exacte, le résultat observé avant sécurisation et le résultat observé après.

Le rapport et la démonstration. L’analyse écrite, qui relie chaque axe de sécurité à la preuve correspondante, et une démonstration en direct devant la classe.

12. Critères de réussite et contraintes

Critère de réussite. La solution doit être fonctionnelle, testée, documentée et présentée avec des preuves techniques. Toute affirmation du rapport devra être appuyée par une requête que n'importe qui dans la salle peut rejouer : un axe de sécurité n'est considéré comme traité que lorsque le test correspondant de la section 9 a été observé et consigné.

Exécution locale. L'application s'exécute sur la machine de travail. Dans son état antérieur à la phase de sécurisation, elle présente des faiblesses volontairement conservées à des fins de démonstration et ne doit en aucun cas être exposée sur un réseau.

Données fictives. Comptes, notes, certifications et messages sont entièrement fictifs et produits par le jeu de données initial. Aucune donnée personnelle réelle n'est utilisée.

Choix des algorithmes. Les algorithmes retenus privilégient la lisibilité du mécanisme et l'absence de dépendance à un service externe. Un système de production retiendrait une fonction de hachage de mots de passe à coût mémoire, ce qui sera précisé dans le rapport.